

```

0000 separate (IDL.SEM_PHASE) use NEWSNAM; use VIS_UTIL; use PRE_FCNS;
function HASH_NODE_HASH (NODE_HASH : NODE_HASH_TYPE;
PG) - 79 * INTEGER (NODE.LN) );
begin
    HASH_CODE := HASH_CODE mod NODE_HASH.A'LENGTH;
    return HASH_CODE;
end HASH_NODE_HASH;

-- procedure INSERT_NODE_HASH (NODE_HASH : in out NODE_HASH_TYPE;
-- NEW_NODE : TREE;
-- OLD_NODE : TREE;
-- HASH_INDEX : NATURAL := HASH_NODE_HASH (NODE_HASH, OLD_NODE);
-- HASH_CHAIN : TREE := NODE_HASH.A (HASH_INDEX);
-- NEW_HASH_CHAIN : TREE := MAK
-- E (DN_LIB_INFO);
-- begin
--     D (XD_SHORT, NEW_HASH_CHAIN, HASH_CHAIN);
--     D (XD_PRIMARY, NEW_HASH_CHAIN, OLD_NODE);
--     D (XD_SECONDARY, NEW_HASH
-- CHAIN, NEW_NODE);
--     NODE_HASH.A (HASH_INDEX) := NEW_HASH_CHAIN;
-- end INSERT_NODE_HASH;

-- procedure SE
-- ARCH_NODE_HASH (NODE_HASH : in out NODE_HASH_TYPE;
-- NODE : in out TREE;
-- HASH_INDEX : NATURAL := HASH_NODE_HASH (NODE_HASH, NODE);
-- HASH_CHAIN : TREE := NODE_HASH.A (HASH_INDEX);
-- begin
--     while HASH_CHAIN /= TREE_VOID loop
--         if D (XD_PRIMARY, HASH_CHAIN) = NODE then
--             NODE := D (XD_SECONDARY, HASH_CHAIN);
--             exit;
--         end if;
--         HASH_CHAIN := D (XD_SHORT, HASH_CHAIN);
--     end loop;
-- end SEARCH_NO
0010 DE_HASH;
LD_NODE : constant TREE := NODE;
begin
    NODE := COPY_NODE (NODE);
    INSERT_NODE_HASH (NODE_HASH, NODE, OLD_NODE);
end REPLACE_NODE;

-- procedure SUBSTITUTE_ATTRIBUTES (NODE : in out TREE;
-- NODE_HASH : in out NODE_HASH_TYPE;
-- H_IN : H_TYPE;
-- is use IDL_TBL;
-- H : H_TYPE;
-- NS_SIZE loop
--     ATTRIBUTE := DABS (I, NODE);
--     OLD_ATTRIBUTE := ATTRIBUTE;
--     SUBSTITUTE (ATTRIBUTE, NODE_HASH, H);
--     if AT
-- TRIBUTE /= OLD_ATTRIBUTE then
--         DABS (I, NODE, ATTRIBUTE);
--     end if;
-- end loop;
-- end SUBSTITUTE_ATTRIBUTES;

-- procedure SUBSTITUTE_ATTRIBUTES_ON_NODE_COPY (NODE : in out TREE;
-- NODE_HASH : in out NODE_HASH_TYPE;
-- H_IN : H_TYPE;
-- is use IDL_TBL;
-- OLD_NODE : constant TREE := NODE;
-- OLD_ATTRIBUTE : TREE;
-- ATTRIBUTE : TREE;
-- begin
--     for
-- I in 1 .. N_SPEC (NODE.TY).NS_SIZE loop
--         ATTRIBUTE := DABS (I, NODE);
--         OLD_ATTRIBUTE := ATTRIBUTE;
--         SUBSTITUTE (ATTRIBUTE, NODE_HASH,
-- H);
--         if ATTRIBUTE /= OLD_ATTRIBUTE then
--             if NODE = OLD_NODE then
--                 NODE := COPY_NODE (NODE);
--             end if;
--             DABS (I, NODE, ATTRIBUTE);
--         end if;
--     end loop;
-- end SUBSTITUTE_ATTRIBUTES_ON_NODE_COPY;

0020 TE (NODE : in out TREE;
-- NODE_HASH : in out NODE_HASH_TYPE;
-- H_IN : H_TYPE;
-- is use IDL_TBL;
-- OLD_NODE : constant TREE := NODE;
-- H : H
-- TYPE;
-- renames H_IN;
-- begin
--     if NODE_HASH.LIMIT > 0 then
--         NODE_HASH.LIMIT := NODE_HASH.LIMIT - 1;
--         else
--             PUT_LINE ("!! RUNAWAY LOOP I
-- N GENERIC SUBSTITUTION");
--             raise PROGRAM_ERROR;
--         end if;
--         if (NODE.PT = HI or NODE.PT = S) or else
-- DABS (0, NODE).N
-- SIZ = 0) then
--             return;
--         end if;
--         SEARCH_NODE_HASH (NODE_HASH, NODE);
--         if NODE /= OLD_NODE then
--             return;
--         end if;
--         case NODE.TY is
--             when DN_LIST => DN_BLOCK_MASTER | CLASS_TYPE_DEF | CLASS_SEQUENCES | CLASS_NAMED_ASSOC | CLASS_NAMED_REP | DN_RECORD_REP | DN_USE | CL
-- ASS_VARIANT_ELEM | DN_ALIGNMENT | DN_VARIANT_PART | DN_COMP_LIST | DN_INDEX | CLASS_USED_NAME | CLASS_NAME_EXP | CLASS_EXP_EXP | CLASS_CONSTRA
-- INT | CLASS_CHOICE | CLASS_HEADER | CLASS_UNIT_DESC | CLASS_MEMBERSHIP_OP | CLASS_SHORT_CIRCUIT_OP | CLASS_COMP_REP_ELEM => SUBSTITUTE (NODE_HASH,
-- ES_ON_NODE_COPY (NODE, NODE_HASH, H);
--             when DN_TYPE_DECL => declare SOURCE_NAME : TREE := D (AS_SOURCE_NAME, NODE);
--             DERIVED_ID_LIST := SEQ_TYPE;
--             DERIVED_ID := TREE;
--             begin
--                 REPLACE_SOURCE_NAME (SOURCE_NAME, NODE_HASH, H);
--                 SUBSTITUTE_ATTRIBUTES_ON_NODE_COPY (NODE, NODE_HASH, H);
--             end;
--             if D (AS_TYPE_DEF, NODE).TY = DN_DERIVED_DEF then
--                 DERIVED_ID_LIST := LIST (D (AS_TYPE_DEF, NODE));
--                 while not IS_EM
-- PTY (DERIVED_ID_LIST) loop
--                     POP (DERIVED_ID_LIST, DERIVED_ID);
--                     REPLACE_SOURCE_NAME (DERIVED_ID, NODE_HASH, H);
--                     end loop;
--                     end if;
--                     SUBS
-- TITUTE_ATTRIBUTES_ON_NODE_COPY (NODE, NODE_HASH, H);
--                     when DN_SUBTYPE_DECL => declare SOURCE_NAME : TREE := D (AS_SOURC
-- E_NAME, NODE);
--                     begin
--                         REPLACE_SOURCE_NAME (SOURCE_NAME, NODE_HASH, H);
--                         SUBSTITUTE_ATTRIBUTES_ON_NODE_COPY (NODE, NODE_HASH, H);
--                     end;
--                     when CLASS_UNIT_DECL | DN_TASK_DECL | CLASS_SIMPLE_RENAME_DECL => declare SOURCE_NAME : TREE := D (AS_SOURCE_NAME, NODE);
--                     begin
--                         REPLACE_SOURCE_NAME (SOURCE_NAME, NODE_HASH, H, NODE);
--                         SUBSTITUTE_ATTRIBUTES_ON_NODE_COPY (NODE, NODE_HASH, H);
--                     end;
--                     when CL
-- ASS_OBJECT_DECL => declare SOURCE_NAME_S : TREE := D (AS_SOURCE_NAME_S, NODE);
--                     SOURCE_NAME_LIST := SEQ_TYPE := LIST (SOURCE_NAME_S);
--                     SOURCE_NAME := TREE;
--                     TYPE_KIND := NODE_NAME := D (AS_TYPE_DEF, NODE).TY;
--                     begin
--                         while not IS_EMPTY (SOURCE_NAME_LIST) loop
--                             POP (SOURCE_N
-- AME_LIST, SOURCE_NAME);
--                             REPLACE_SOURCE_NAME (SOURCE_NAME, NODE_HASH, H, NODE);
--                             if TYPE_DEF_KIND = DN_CONSTRAINED_ARRAY_DEF then
--                                 GEN_PRED
-- EFINED_OPERATORS (D (SM_OBJ_TYPE, SOURCE_NAME), H);
--                             end if;
--                             end loop;
--                             SUBSTITUTE_ATTRIBUTES_ON_NODE_COPY (NODE, NODE_HASH, H);
--                             end;
--                             when CLASS_USED_OBJECT => declare OLD_DEFN : constant TREE := D (SM_DEFN, NODE);
--                             DEFN := TREE := OLD_DEFN;
--                             EXP_TYPE := TREE :=
-- D (SM_EXP_TYPE, NODE);
--                             begin
--                                 SUBSTITUTE (DEFN, NODE_HASH, H);
--                                 if DEFN = OLD_DEFN then
--                                     SUBSTITUTE (EXP_TYPE, NODE_HASH, H);
--                                 end;
--                                 when CLASS_DSCRMT_PARAM_DEC
-- L | DN_NUMBER_DECL | DN_EXCEPTION_DECL | DN_DEFERRED_CONSTANT_DECL => declare SOURCE_NAME_S : TREE := D (AS_SOURCE_NAME_S, NODE);
--                                 SOURCE_NAME := TREE := LIST (SOURCE_NAME_S);
--                                 SOURCE_NAME := TREE;
--                                 begin
--                                     while not IS_EMPTY (SOURCE_NAME_LIST) loop
--                                         POP (SOURCE_NAME
-- LIST, SOURCE_NAME);
--                                         REPLACE_SOURCE_NAME (SOURCE_NAME, NODE_HASH, H, NODE);
--                                         end loop;
--                                         SUBSTITUTE_ATTRIBUTES_ON_NODE_COPY (NODE, NODE_HASH, H);
--                                         end;
--                                         when DN_PRAGMA => declare USED_NAME_ID : TREE := D (AS_USED_NAME_ID, NODE);
--                                         begin
--                                             USED_NAME_ID := COPY_NODE (U
-- SED_NAME_ID);
--                                             if D (SM_DEFN, USED_NAME_ID) /= TREE_VOID then
--                                                 SUBSTITUTE_ATTRIBUTES_ON_NODE_COPY (NODE, NODE_HASH, H);
--                                             end if;
--                                             when CLASS_NON_TASK => if D (SM_BASE_TYPE, NODE) /= NODE then
--                                                 SUBSTITUTE_ATTRIBUTES_ON_NODE_COPY (NODE, NODE_HASH, H);
--                                             end if;
--                                             when DN_ROOT => CLASS_BOOLEAN | DN_NIL | DN_SOURCELINE | DN_ERROR | DN_HASH | DN_VOID | DN_SUBPROGRAM_BODY | DN_PACKAGE_BODY | DN_TAS
0050 K_BODY | DN_SUBUNIT | CLASS_STM_ELEM | CLASS_TEST_CLAUSE_ELEM | CLASS_ITERATION | CLASS_ALTERNATIVE_ELEM | CLASS_CONTEXT_ELEM | DN_COMPILATION |
-- DN_COMPILATION_UNIT | DN_UNIVERSAL_INTEGER | DN_UNIVERSAL_FIXED | DN_UNIVERSAL_REAL | DN_USER_ROOT | DN_TRANS_WITH .. DN_NULLARY_CALL => PUT_LIN
-- E ("GEN_SUBS.SUBSTITUTE : NOEUD INVALIDE EN COPIE GENERIQUE " & NODE_NAME'IMAGE (NODE.TY) );
-- raise PROGRAM_ERROR;
--         when DN_TXTREP | DN_NUM_V
-- AL | DN_SYMBOL_REP | CLASS_DEF_NAME | DN_NULL_COMP_DECL | DN_TASK_SPEC | CLASS_PRIVATE_SPEC | DN_INCOMPLETE | DN_REAL_VAL => null;
--         when
-- DN_VIRGIN => PUT_LINE ("GEN_SUBS.SUBSTITUTE : UN NOEUD NON INITIALISE");
--         raise PROGRAM_ERROR;
--         end case;
--         if NODE /= OLD_NODE then
--             I
-- NSERT_NODE_HASH (NODE_HASH, NODE, OLD_NODE);
--         end if;
--     end SUBSTITUTE;
-- end GEN_SUBS;

```